

stable-fips203-mlkem-kem-keygen-k4

FIPS 203 Alg.19 ML-KEM-1024 KEM KeyGen 实现方案 (k=4)

ascendc 工程 (定型交付)

2026-07-14

摘要

本文档描述 **定型交付算子** `examples/stable/stable-fips203-mlkem-kem-keygen-k4/` 在 **Atlas A2 / Ascend910B4** 上以 AscendC 实现 FIPS 203 **Algorithm 19** ML-KEM.KeyGen() (参数集 `ml_kem_1024`, $K = 4$), 经 **Alg.16** KeyGen_internal 调用 **Alg.13** K-PKE.KeyGen, 并在设备侧完成 $H(ek)$ 与 z 的拼接。

晋级 (2026-07-14): 自 `examples/incubating/exp-fips203-mlkem-kem-keygen-k4/` 复制晋级 (# 交付 #); 预研副本保留。踩坑与强制同步见 §6。

行为对照 (只读; 禁止把探针当编译依赖): `ascendc-tests/pass-fix-f203-alg19-kem-keygen-device-k4/` (CPU+SIM PASS; SIM Total tick $\approx$ **713227**)。定稿原理: `docs/notes/F203-KEM-Alg19-KeyGen设备全链技术总结.md`。PKE 主体行为对齐 `examples/stable/stable-fips203-mlkem-pke-keygen-k4/` (Alg.13, 2 launch, SIM $\approx$ 542k)。

生产 I/O (强制):

- **输入**: `input/seed_d.bin` (4B uint32 LE, 约定 SEED_D=20260619) + 静态 NTT LUT (与 seed 无关)。
- **输出**: 仅 `ek_kem.bin` (1568 B) 与 `dk_kem.bin` (3168 B, **liboqs** 展开)。
- d/z 仅在 device UB 生成与消费, **禁止** D2H / 落盘。

设备 Launch (强制): 与 device 探针相同——**恰好 2 次** (prep || mmad+KEM 尾)。**禁止** correctness 探针式第 3 launch `f203_kem_kg_finish`; **禁止** 为 $H(ek)/z$ 单独 `aclrtSynchronizeStream` + 再 launch。

自包含原则 (与 device 探针 CMake 策略不同): AscendC / Host 源码 **全部 vendored** 于本目录; 唯一允许的外部代码依赖: `library/shared/`。**禁止** 编译期 `#include ascendc-tests/`、其它 `examples/`、`frozen/`。

写法对齐 `stable-fips203-mlkem-pke-keygen-k4-实现方案-customspec.tex`。

目录

1 工程约束 (自包含)	2
1.1 允许的外部依赖	2
1.2 禁止的外部依赖与不安全路径	2
1.3 相对 device 探针的落点差异	2
1.4 唯一交付路径 (默认即全量)	2
1.5 目录布局 (计划; 写码阶段落地)	3

目录	2
2 数学语义 (Alg.19 → Alg.16 → Alg.13)	3
2.1 参数集	3
2.2 Algorithm 19 (对外)	4
2.3 Algorithm 16 (internal)	4
2.4 liboqs 展开 I/O (验收锁定)	4
2.5 Algorithm 13 分段 (设备)	4
3 编排与 Launch	4
3.1 相对 correctness (3 launch)	5
4 AI Core 规模	5
5 GM 布局与零额外搬运	5
5.1 KemKgTailFused 步骤 (汇合后的约定 AIV)	6
6 踩坑与强制同步 (2026-07-13 debug; 换 Agent 必读)	6
6.1 过程摘要	6
6.2 强制条款 (实现时违反即视为未完成)	7
6.3 推荐压测脚本口径 (写码后)	7
7 旁路 A (test-only)	7
8 全量 API 清单 (查阅 CANN 9.0.0)	8
8.1 设备哈希 (非 CANN 矢量 API; 工程共享库)	9
8.2 评估过但未采用	9
9 数学步骤 → API 覆盖率	9
10 验证	10
10.1 Golden 角色	10
10.2 生产验收 (默认; 写码完成后)	10
10.3 Gate 建议 (写码阶段, 非本规格交付物)	10
11 性能 (SIM 910B4 参考)	11
12 写码衔接 (本规格确认后)	11

1 工程约束（自包含）

1.1 允许的外部依赖

路径	用途
library/shared/shake_x of_kernel/	Prep SampleNTT / PRF (ProcessInline)
library/shared/keccak_ f1600_kernel/	Keccak-f[1600]: Prep $G(d k)$ (SHA3-512) 与 KEM 尾 Sha3OneShot (SHA3-256: 派生 z 、 $H(ek)$)
library/shared/	公共头（如 shake_general.h、 fips203_device_sha3.hpp）
CANN / tikicpulib	工具链与仿真（非业务源码）

1.2 禁止的外部依赖与不安全路径

- 运行时 / 编译期依赖 `ascendc-tests/`（含 device / correctness 探针）、`frozen/`、其它 `examples/*/`（一次性 vendor 除外）。
- 子进程调用 `stable /` 探针 `run.sh` 再拼 KEM。
- Host `tiny_sha3 / liboqs` 参与生产路径（KAT 脚本除外）。
- correctness 式 Launch-3; Launch-2 后对 `ekPKE/dkPKE` 做 $D2H \rightarrow H2D$ 再算 H/z 。
- 将 d 、 z 、 $H(ek)$ 、 $\hat{A}$ 、 $\hat{s}$ 、 ρ 等中间态写入 `input//output/` 作为交付契约。

1.3 相对 device 探针的落点差异

项	device 探针	本 exp（锁定）
PKE 源码	CMake STABLE_ROOT 编译 期引用	本目录 <code>prep/+compute/ vendored</code>
KEM 尾	<code>kem/</code> + 宏 <code>F203_KEM_KEYGEN_TAIL=1</code>	同宏语义；尾实现 vendored 于 <code>kem/</code>
Launch / I/O / GM	2 launch; ek 1568 / dk 3168	同契约

1.4 唯一交付路径（默认即全量）

组件	锁定配置
Prep $\hat{A}$	<code>F203_AHAT16_BLOCK_DIM=2</code> , 双 AIV 并行 <code>BuildAHat16ShardWithUb</code>

Prep Alg7/CBD	与 stable PKE KeyGen 同生产宏 (rej / CBD _{η₂} 向量路径)
Compute 2s1e	F203_STAGE1_SPLIT=1, mixPass=0; NTT S1-S3 禁 Gather
行 18-21	HAT_LINE18_DOT_ONLY=0, HAT_BYTE_ENCODE=1, F203_KEYGEN_EK_PKE=1
KEM 尾	F203_KEM_KEYGEN_TAIL=1: KemKgTailFused 内嵌 于 Launch-2; 须先双 AIV 汇合 (见 §6), 再由约定 AIV 执行
Host	2 launch; seed+LUT → 仅 ek_kem/dk_kem

禁止将关尾 (F203_KEM_KEYGEN_TAIL=0)、关 ByteEncode、分段 Host 入口等作为默认验收; 调试对照须显式覆盖 env, 并在 run.sh 头注释标「非默认」。

1.5 目录布局 (计划; 写码阶段落地)

```
stable-fips203-mlkem-kem-keygen-k4/
  stable-fips203-mlkem-kem-keygen-k4-实现方案-customspec.tex/.pdf
  main_kem_keygen.cpp          # Host: 2 launch + dk_kem_gm
  f203_kem_kg_layout.h         # I/O 常量 (1568/3168 偏移)
  f203_keygen_prep_entry.cpp    # Launch-1 (vendored PKE prep)
  prep/                         # vendored Alg.13 行 3-15
  compute/mmad_custom.cpp       # vendored Alg.13 行 16-21 + 尾钩子
  kem/
    f203_kem_kg_derand_ub.hpp   # DerandZFromSeedD
    f203_kem_kg_tail_fuse.hpp   # KemKgTailFused (无 ek 独立拷贝)
  cmake/keygen/CMakeLists.txt
  scripts/                      # golden / prepare / verify / KAT 胶水
  run.sh
  SELF_CONTAINED.md
  STATUS.md
```

2 数学语义 (Alg.19 → Alg.16 → Alg.13)

2.1 参数集

ml_kem_1024: $n = 256$, $q = 3329$, $k = 4$ 。(文档勿与历史笔误「ML-KEM-768」混用; 本仓 PKE/KEM 交付均为 1024 档。)

2.2 Algorithm 19 (对外)

$d \leftarrow \mathcal{S}\{0, 1\}^{256}$ Launch-1 prep UB: DerandFromSeedD(seed_d)
 $z \leftarrow \mathcal{S}\{0, 1\}^{256}$ Launch-2 尾 UB: DerandZFromSeedD(seed_d)
 $(ek, dk) \leftarrow \text{ML-KEM.KeyGen_internal}(d, z)$

可复现验收: Host 仅提供 4B seed_d; 不提供 64B 外部 $d||z$ (旁路 A 仅 test-only, 见 §7)。

2.3 Algorithm 16 (internal)

$(ek_{\text{PKE}}, dk_{\text{PKE}}) \leftarrow \text{K-PKE.KeyGen}(d)$ Alg.13, Launch-1+2 主体
 $ek \leftarrow ek_{\text{PKE}}$
 $dk \leftarrow dk_{\text{PKE}} || H(ek) || z$

其中 $H = \text{SHA3-256}$ 。

2.4 liboqs 展开 I/O (验收锁定)

代数最小 $dk_{\text{PKE}} || H || z$ (1600 B) 不是本仓交付布局。对拍 OQS_KEM_ml_kem_1024 时:

$ek_kem = ek_PKE$ // 1568 B
 $dk_kem = dk_pke$ (1536) || ek (1568) || $H(ek)$ (32) || z (32) // 3168 B

偏移 (字节): dk_{PKE} [0, 1536); ek [1536, 3104); H [3104, 3136); z [3136, 3168)。

2.5 Algorithm 13 分段 (设备)

FIPS 行	设备段	Launch
3–15	f203_keygen_prep: $\hat{A}$ 、 ρ/σ 、 $\hat{s}/\hat{e}$ CBD	L1
16–20	mmad_custom: NTT、内积、 ByteEncode ₁₂	L2
21	FuseEkPke: $ek_{\text{PKE}} = ek_{\text{polyvec}} \rho$	L2 末 AIV0
Alg.16 尾	KemKgTailFused: z 、 $H(ek)$ 、拼 dk_kem	L2 末 AIV0 (同次)

3 编排与 Launch

aclInit / CreateStream

Launch-1: f203_keygen_prep // AIV_ONLY, blockDim=2

```

seed_d → d → / →  $\hat{A}$ ,  $\hat{s}/\hat{e}$ , ...
Launch-2: mmad_custom          // MIX_AIC_1_2, blockDim=1
Alg.13 行 16-21 → ek_pke_gm, dk_pke_gm
尾 (AIV0, F203_KEM_KEYGEN_TAIL=1) :
    DerandZ → H(ek) → 写 dk_kem_gm[3168]
    ek_kem    ek_pke_gm (同址, 零拷贝)
aclrtSynchronizeStream          // 仅一次, L2 后
D2H: ek_kem.bin, dk_kem.bin
aclFinalize

```

3.1 相对 correctness (3 launch)

项	correctness-k4	本方案 / device
Launch 数	prep mmad kem_finish	prep mmad+tail
中间 sync	L2 后 + L3 后	仅 L2 后 (同 stable)
ek_kem	可独立拷贝	与 ek_pke 同指针
SIM tick	≈742k	目标 ≈stable+ α (device ≈713k)

4 AI Core 规模

段	核类型 / blockDim / 角色
Prep (L1)	KERNEL_TYPE_AIV_ONLY; blockDim=2; 双 AIV 各 8 poly 分片 $\hat{A}$; block0 独占 PRF+CBD
Compute (L2)	KERNEL_TYPE_MIX_AIC_1_2; blockDim=1; 1 AIC + 2 AIV; NTT/MMAD/Encode
KEM 尾	同 L2; Encode 后 双 AIV 汇合 再由约定 AIV 执行 Fuse+ 尾 (SIM: subBlockID==0; CPU: ==1; 见 §6); 不新增 AI Core、不增 blockDim

串行占用 **1 颗 AI Core** (与 stable PKE KeyGen 同)。CPU [SUCCESS][AIC_*] 为 tikicpu 拓扑 artifact; 以 SIM profile_*.toml 为准。

选型理由: 复用已验收 PKE KeyGen 拓扑; KEM 增量仅尾段标量 SHA3 + 字节拼接, 不值得为尾段单独开 AIV_ONLY launch (correctness 已否决)。

5 GM 布局与零额外搬运

GM 缓冲	尺寸	用途
-------	----	----

seed_d_gm	4B (旁路 A: 64B)	L1 读; z 派生再读
a_hat...rho...	同 stable	L1 写 / L2 读
ws	LUT+workspace	
ek_pke_gm	1568	L2 写; = ek_kem_gm 别名
dk_pke_gm	1536	L2 写; 尾段读
dk_kem_gm	3168	仅尾段写 (唯一新增 KEM GM)

不新增: h_gm、z_gm、独立 ek_kem 缓冲 (H/z 全程 UB)。

5.1 KemKgTailFused 步骤 (汇合后的约定 AIV)

1. DerandZFromSeedD (或旁路 A 读 [32:64]) $\rightarrow z[32]$ UB。
2. 自 ek_pke_gm 读入 UB $\rightarrow \text{Sha3OneShot} \rightarrow H[32]$ UB。
3. 顺序写 dk_kem_gm: $\text{dk}_{\text{PKE}} \parallel \text{ek} \parallel H \parallel z$ 。
4. 省略 $\text{ek}_{\text{kem}} \leftarrow \text{ek}_{\text{PKE}}$ 拷贝 (同址)。

同步 (强制, 2026-07-13 订正):

- ByteEncode 由 双 AIV 分片写 ek_out/sk_out; FuseEkPke/KemKgTailFused 须读完整 1536 B sk_out。
- 禁止仅靠本核 PipeBarrier/KYBER_PIPE_ALL 后立刻做尾 (本核屏障不能等待对端 AIV 写完 GM)。
- 设备 / SIM: 双 AIV 在 Encode 后均调用 AscendC::SyncAll</*isAIVOnly=*/true>() (硬同步, A2 支持; 见 CANN API §2.3.7.2.3 / 查阅索引), 再由 subBlockID==0 做 FuseEkPke+尾。
- CPU (ASCENDC_CPU_DEBUG): tikicpu 常串行 AIV0 $\rightarrow$ AIV1; 对 AIV0 做硬 SyncAll 易死等。须改由 subBlockID==1 (后执行的 AIV) 做 Fuse+ 尾, 或等价保证「两片 Encode 均已落 GM 后再读 sk_out」。
- KYBER_PIPE_ALL 须定义为真实 PipeBarrier<PIPE_ALL> (禁止 CPU 下空操作); 尾段前后再显式 PipeBarrier。

详证与误判史见 §6。

6 踩坑与强制同步 (2026-07-13 debug; 换 Agent 必读)

6.1 过程摘要

1. 曾按本规格写码并一度声称 CPU+SIM PASS, 随即 # 验收 # 复制晋级 stable —— 流程错误: incubating 偶发对拍未根治不得晋级。

2. 偶发: `ek_kem` PASS / `dk_kem` FAIL ($\max \approx 254$), 或 `ek[768:]` 错。初判归咎 `KYBER_PIPE_ALL` 在 CPU 为空操作, 仅加本核 `PipeBarrier` 后短压测曾绿, **仍不够**。
3. 精确定位 (同二进制连跑 + 分段比对): FAIL 时 **仅** `dk_kem` 前 1536 B (`dk_PKE`) 出错, 首坏偏移常为 **1152** ($= 3 \times 384$, 末 poly, 属 AIV1 分片); `ek_kem`、`dk` 内 `ek` 段、 $H(ek)$ 、 z **全对**。⇒ Tail 在 AIV1 写完 `sk_out` 前就拷贝了 `dk_PKE`。
4. 残留掩盖: 连续跑同一 seed 时, GM 上旧的正确 `sk_out` 可使「抢跑拷贝」仍对拍通过。验收须**清零** `output/` 或强制重编后多轮压测 (建议 CPU ≥ 40 次), 不可偶发 PASS 即晋级。
5. 用户裁决 (2026-07-13): 删除 `stable` 与 `incubating` **全部实现**, 仅留本规格; 回家 Agent 按规格重写; `debug` 写入当日 `qa/` 与 `AGENT_HANDOFF.md`。

6.2 强制条款 (实现时违反即视为未完成)

条款	要求
双 AIV 汇合	Encode 后、Fuse/Tail 前: SIM/设备 <code>SyncAll<true></code> ; CPU 用 AIV1 做尾或其它 已证明 无死锁方案
禁侥幸验收	禁止「跑通一次就晋级」; 须 CPU 多轮 + SIM; 建议清 零 <code>output</code> 防残留
禁空屏障	<code>KYBER_PIPE_ALL</code> 不得在 <code>ASCENDC_CPU_DEBUG</code> 下为空
禁过早 <code>stable</code>	仅 <code>incubating</code> CPU $\times N$ +SIM 稳定绿后, 用户 # 交付 ## 验收 # 再复制晋级; 出错先改 <code>incubating</code>
禁抄 <code>frozen</code>	含 <code>examples/frozen/</code> 、 <code>ascendc-tests/frozen/</code>
基线只读	<code>device</code> 探针可对照 I/O/行为; 禁止 编译期依赖其源码路 径

6.3 推荐压测脚本口径 (写码后)

```
# 强制重编 + 单次
KEM_KEYGEN_FORCE_REBUILD=1 bash run.sh -r cpu -v Ascend910B4
# 同二进制多轮 (每轮可先 dd 清零 output/*.bin)
# ... kernel + gen_data + verify_kem ...
SIM_DIRECT=1 bash run.sh -r sim -v Ascend910B4
# 用例根目录不得残留 stray core*.dump / profile*_log*.toml
```

7 旁路 A (test-only)

`KEM_KG_EXT_SEED=1`: `seed_d_gm` 扩为 64 B ($d||z$ 外部给定), `prep` 走 `extseed` 入口; 生产默认关闭。用于 `liboqs` 批量 KAT; 不得改变默认 `run.sh` 契约。

8 全量 API 清单 (查阅 CANN 9.0.0)

说明：PKE 主体 (prep / NTT / MMAD / ByteEncode₁₂ / FuseEkPke) API 与 exp-fips203-mlkem-pke-keygen-k4 / device 探针同集，已在 library/documents/CANN-AscendC算子开发接口参考-查阅索引.md 有记录；下表列本用例须显式锁定的主路径 API (含 KEM 尾)。分类按 PDF 第 2 章；在线 ID 为社区 atlasascendc_api_07_*。

API	分类	在线 ID	A2	输入/输出 (本用例)	备注
GlobalTensor	2.2	07_0007	✓	GM 描述	SetGlobalBuffer
LocalTensor	2.2	07_0006	✓	UB 描述	尾段字节缓冲
GetBlockIdx	2.3 系统	07_0185	✓	核索引	Prep 分片
GetSubBlockIdx	2.3 系统	07_0186 邻	✓	AIC/AIV 子块	Fuse/尾: SIM 用 0; CPU 用 1 (§6)
KERNEL_TYPE_AIV_ONLY	Utils	编程指南	✓	L1	Prep
KERNEL_TYPE_MIX_AIC_1_2	Utils	编程指南	✓	L2	1 AIC : 2 AIV
TPipe	2.3 资源	07_0108	✓	管道	尾可独立小 Pipe
InitBuffer	2.3 资源	07_0110	✓	UB 分配	尾段
TQue/TBuf	2.3 Pipe	07_0137	✓	队列/临时区	Prep/Compute
DataCopy	2.3.1	07_0101	✓	uint8/int32 GM↔UB	尾: ek 读入、 dk_kem 写出
DataCopyPad	2.3.1	07_0265	✓	可选跨步	尾主路径不用 (连续块)
ShiftRight	2.3.3	07_0059	✓	int32	Stage1 $hi=v\gg 6$
Muls	2.3.3	07_0055	✓	int32	Stage1 $hi\times 64$ 等
Adds/Add/Sub	2.3.3	07_0054/35/36	✓	int32	NTT/mod/Encode 链
Duplicate	2.3.3	07_0041 邻	✓	填充	Prep/对齐块
CrossCoreSetFlag	同步	API 列表	✓	AIC↔AIV	L2 NTT 流水
CrossCoreWaitFlag	同步	API 列表	✓	同上	与 vec-k4-v2 同构
SyncAll	同步	§2.3.7.2.3	✓	硬同步 SyncAll<true>()	Encode 后双 AIV 汇合 (§6); isAIVOnly=true (AIC 已返回)
PipeBarrier	同步	常用	✓	PIPE_ALL 等	本核 MTE/V; 不能代替双 AIV 汇合

API	分类	在线 ID	A2	输入/输出（本用例）	备注
Gather	2.3.3	07_0071 邻	✓	—	NTT S1–S3 禁用； Encode 后另议

8.1 设备哈希（非 CANN 矢量 API；工程共享库）

符号	契约
F203SeDeviceKeccak:: Sha30neShot	library/shared/keccak_f1600_kernel/fips203 _device_sha3.hpp; __aicore__ one-shot; mdlen=32 (z 、 $H(ek)$) /64 (Prep G)。日后可换第 三方 AscendC SHA3，勿改 layout 偏移与域分离 串。

8.2 评估过但未采用

方案	结论
correctness 第 3 launch kem_finish	多一次 sync+launch；SIM 约 742k； 否决 （device 已 PASS 2 launch）
Host 拼 dk _{kem} / Host SHA3	违反设备全链； 禁止
独立 ek_kem GM 拷 贝	与 ek _{PKE} 字节恒等；零拷贝更优
mmad fork (mmad_custom_kem.cpp)	device P1 已废弃；本 exp vendored 单份 mmad_custom + 宏钩子
仅本核 PipeBarrier 后 AIV0 做尾	否决 ：无法等待 AIV1 写完 sk_out；见 §6
CPU 下空操作 KYBER_PIPE_ALL	禁止
未稳定绿即复制 stable	禁止 （用户 2026-07-13 明确）
NTT S1–S3 内 Gather	Tag5T 禁令；沿用 PKE KeyGen
从 frozen/ 抄 KEM/PKE	Rule 禁止

9 数学步骤 → API 覆盖率

数学步骤	主路径	覆盖	缺口/备注
$G(d k) \rightarrow \rho, \sigma$	SHA3-512 共享库	标量	Keccak 标量核（与 PKE 同）
SampleNTT $\hat{A}$	SHAKE + rej 向量	部分	与 stable Prep 同；Alg7 标量缺口见 PKE 纪要
PRF+CBD $_{\eta_2}$	向量 MTE2/V	高	同 PKE KeyGen
NTT / 内积 / Encode $_{12}$	MIX 向量 +Cube	高	S1-S3 无 Gather；见 MLKEM-NTT 定稿
ek ρ （行 21）	DataCopy	100% 搬运	AIV0 FuseEkPke
Derand z	SHA3-256	标量	UB only
$H(\text{ek})$	SHA3-256 + DataCopy	标量摘 要	1568 B 入 UB 后 one-shot
拼 dk_kem	DataCopy 四段	100% 搬运	无跨 launch

主路径结论：PKE 计算段与 stable 同覆盖率；KEM 增量的密码学哈希为**显式标量缺口**（设备共享 Keccak，非矢量 API）。拼接为纯搬运，**100% DataCopy**。

10 验证

10.1 Golden 角色

Host / liboqs 仅提供合法输入与期望输出（黑盒 oracle）。**禁止**以「与 device / correctness 源码同构」为验收；仅 **I/O 等价**。

10.2 生产验收（默认；写码完成后）

```
cd examples/stable/stable-fips203-mlkem-kem-keygen-k4
bash run.sh -r cpu -v Ascend910B4
SIM_DIRECT=1 bash run.sh -r sim -v Ascend910B4
# 检查 output/ek_kem.bin (1568B) + dk_kem.bin (3168B)
```

对照 oracle（写码阶段）：同 SEED_D 下与 pass-fix-f203-alg19-kem-keygen-device-k4 及/或 fix-f203-alg19-kem-keygen-correctness-k4 的 ek_kem/dk_kem 逐字节一致；可选 bash scripts/liboqs_kem_vs_ascendc.sh（仓库根）。

禁止把一长串与 default 相同的 HAT_*=0/1 写入标准验收命令。

10.3 Gate 建议（写码阶段，非本规格交付物）

Gate	验收
------	----

G1	临时关尾: ek/dk _{PKE} vs stable 同 seed max=0
G3	开尾: ek _{kem} /dk _{kem} vs device/correctness max=0
G4	默认 run.sh CPU+SIM PASS; dump 仅在 OP_PROF_*/dump/
G4b	CPU 同二进制 ≥ 40 轮 verifyek/dk max=0 (建议清零 output 防残留)
G5	旁路 A + liboqs 批量 (可选)
G6	SIM tick 量级 $\leq$ device + 50k (参考 $\approx 713k$; 远超则 profile)

11 性能 (SIM 910B4 参考)

- stable PKE KeyGen: Total tick ≈ 542339 (prep $\approx 462k$ + mmad $\approx 80k$)。
- device KEM KeyGen (本基线): Total tick ≈ 713227 (P1 宏钩子; 相对早期 mmad fork $\approx +1.8\%$)。
- 本 stable 验收目标: 与 device 同量级 (实测 $\approx 707k$); 远超视为回归。
- $\sim 15s$ 仅为 Tag5T NTT 全流程性能定标, 不套用本用例防挂死预算; 预算写进本用例 run.sh (建议默认 900s 量级, 与 device 同)。

12 写码衔接 (本规格确认后)

1. 用户确认本 customs spec (含 §6)。
2. 用户明确 【预研】 / 「可以写代码」并指明本路径后, 按 .cursor/skills/pre-research/S KILL.md 在本目录从零落地 vendored 源码 (当前目录无实现可抄)。
3. 登记表: docs/specs/fips203-mlkem1024-kem-keygen-baseline-registry.md。
4. incubating 经验收阶梯 (含 G4b) 稳定后, 用户 # 交付 #/# 验收 # 再复制晋级 examples/stable/stable-fips203-mlkem-kem-keygen-k4/; 禁止未绿先晋级、禁止 stable 与 incubating 双轨各改各的。